HuntAssist Stage 2 Implementation Report

EECS 3311 Software Design · Fall 2026 · 11 November 2026

1. Project overview

HuntAssist is a Java 21 workbench for reviewing authorized HTTP captures. It imports Burp XML and HAR, applies workspace scope, and stores route metadata. A browser interface and a Picocli interface call the same services. The workbench provides scoped references, defensive XSS mitigation observations, remediation recommendations, and coverage history.

Java determines which recommendations are eligible and ties them to stored observations. When enabled, a local Ollama model orders the eligible action codes using a bounded set of boolean signals. The application validates the full order and renders its own remediation text. Without a usable model response, it supplies the deterministic order with a visible status.

2. Feature to implementation mapping

Java paths in this section are relative to `src/main/java/ca/yorku/eecs3311/huntassist/`. Workspace classes are in `application/`, `domain/project/`, and `adapter/persistence/`. Traffic classes are in `evidence/`, and the knowledge and review classes are in `review/`. Controllers and commands are in `presentation/web/` and `presentation/cli/`.

F01 · Workspace and scope · Deterministic

Stage 1 FR-01/02, UC-01. `ProjectService.create` validates the workspace and delegates canonical host rules to `ProgramScope`. `JdbcProjectRepository` saves the project and rules together. The same exclusion-first policy classifies hosts in traffic imports. Pattern roles include Repository and Specification.

F02 · Traffic import and history · Deterministic

Stage 1 FR-02/03/04/17/18, UC-02. `EvidenceService.importTraffic` selects a `TrafficImporter` by content, checks the project scope, and commits a digest, summary, and eligible observations. `BurpXmlTrafficImporter` and `HarTrafficImporter` are interchangeable parsing strategies. A duplicate digest in the same project and profile returns the existing batch.

F03 · Route inventory and observation detail · Deterministic

Stage 1 FR-05/06, UC-04. `EvidenceNormalizer.route` derives a route, and `EvidenceService.inventory` groups its observations. `evidence` supplies per-exchange source indexes, statuses, media types, defensive header flags, and profile labels. The original parameters and application workflow inference were narrowed to sanitized route metadata.

F04 · Defensive reference memory · Deterministic

Stage 1 FR-07/08, UC-03. `KnowledgeService.importNote` uses `NoteParser` to extract UTF-8 Markdown, text, or HTML sections. A changed document creates a version; `search` retrieves current matching sections with provenance. The implementation uses H2 tokens and scores in place of Lucene.

F05 · Pluggable XSS mitigation review · Deterministic

Stage 1 FR-20, UC-06. `VulnerabilityModuleRegistry.require` resolves a category. `XssMitigationModule.assess` uses retained response metadata and the shared `VulnerabilityModule` contract. It recommends defensive checks, including output encoding and review of relevant security headers. It reports applicability explicitly and never treats a header signal as a confirmed finding.

F06 · Evidence-grounded remediation review · Hybrid

Stage 1 FR-10/11/12/19, UC-05. `ReviewService.plan` gathers scoped evidence, relevant local note citations, and coverage. `OllamaRanker.rank` adapts the optional local model. Only a valid permutation of eligible action codes can change the order. `ReviewRepository.saveRun` records action codes, supporting IDs, citations, and run status.

F07 · Import-profile comparison · Deterministic

Stage 1 FR-06/09, UC-07. The interfaces pass two imported labels to `EvidenceService.compare`. The service groups observations by normalized route and reports counts and status sets side by side. These are descriptive differences between existing observations.

F08 · Coverage and decisions · Deterministic

Stage 1 FR-13/14/17, UC-08. `ReviewService.recordCoverage` validates an endpoint, module, and review state. `ReviewRepository.saveCoverage` updates the current state and appends history in one transaction. The planner suppresses suggestions for addressed or dismissed reviews. Coverage is recorded per endpoint and module.

F09 · Question-guided review · Hybrid

Stage 1 FR-15, UC-09. `ReviewService.ask` identifies a supported defensive topic from the question and retrieves matching note sections. It selects eligible actions from the same scoped assessment and optionally applies model ordering. The question stays local to the Java process.

F10 · Privacy, provenance, and audit · Deterministic

Stage 1 FR-11/12/16/17/18/19, UC-02/03/05/08. Import keeps a minimized record and omits bodies and credential values. Evidence, note sections, imports, and review runs have stable IDs. `ProjectStoreLock.acquire` runs before Spring opens H2. CLI and browser show typed, sanitized results. The fixed loopback model adapter sends neither raw capture text nor user notes.

3. Design patterns and class relationships

- **Strategy** — `TrafficImporter.supports/parse` provides two format-specific implementations. `VulnerabilityModule.assess` applies the same extension pattern to defensive checks. Both choices match the Stage 1 design.
- **Adapter** — `WorkbenchController` and `HuntAssistCommand` translate web forms and terminal arguments into shared service calls. `OllamaRanker` translates the narrow `LocalRanker` contract into Ollama chat messages. This preserves the two inbound adapters and narrows the model adapter to loopback.
- **Command** — Picocli dispatches a named operation such as `TrafficCommands.importTraffic` or `PlanCommands.generate` with typed arguments. Each command binds one user request to one service workflow. This is the user-facing Command implementation in place of the planned model-driven tool commands.
- **Facade** — `ReviewService.plan` offers one entry point over evidence, module resolution, knowledge, model ranking, and audit. This makes the same review workflow available to both interfaces.
- **Repository** — `ProjectRepository` isolates scope persistence, `JdbcProjectRepository` implements it, and `ReviewRepository` groups coverage and review writes. This follows the Stage 1 persistence boundary.
- **Specification and Registry** — `ProgramScope.classify` composes host predicates with exclusion precedence. `VulnerabilityModuleRegistry` resolves a module and rejects duplicate IDs at startup. These retain the original scope and extension decisions.

The class and interaction views show the implemented service relationships. Package names are abbreviated in the figures. The Stage 1 diagrams remain in the repository for comparison.

Implemented class relationships

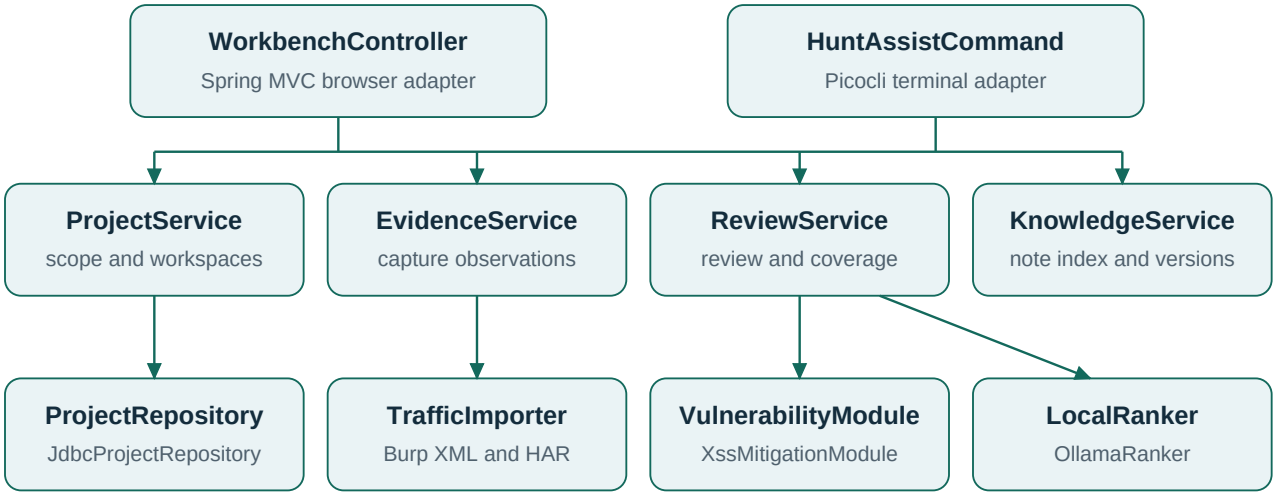

Figure 1. Presentation adapters and the main service and extension interfaces.

Remediation review interaction

1	**GUI or CLI**	request scoped review
2	**ReviewService**	load evidence and references
3	**Module**	select eligible actions
4	**LocalRanker**	order candidate codes
5	**ReviewRepository**	record result and citations

Figure 2. Scope and coverage restrict eligible actions. Failed model validation preserves the Java order.

Implemented use cases

A local reviewer can open a workspace, import captures and notes, inspect observations, compare profiles, request a remediation review or ask a question, and update coverage through either interface. Local Ollama participates only in optional ranking.

4. Changes from Stage 1

- **UC-02/04/07 traffic and comparison** — Raw evidence storage, parameter extraction, GraphQL enrichment, and workflow inference were replaced with a minimized route inventory and profile comparison. Imports retain only fields needed by the review. Batch digests provide deduplication within a profile.
- **UC-03 knowledge** — Individual local Markdown, HTML, and text files are supported. H2 holds the token index and versions in one store. Folder ingestion and live Notion synchronization remain outside this implementation.
- **UC-05/06/09 model and module** — An offline mitigation module replaces payload and exploit-oriented investigation. The model can rank vetted remediation actions after Java has applied scope, retrieval, and coverage. The implementation omits cloud providers and a general model tool loop.
- **UC-08 coverage** — Endpoint/module states replace individual hypothesis, account, and test-outcome records. Coverage and audit history still let users see what was reviewed and suppress completed work.
- **UC-10 interfaces** — Spring MVC and Picocli use the same service methods. Exclusive ownership protects the local store. Commands and equivalent browser flows are shown in the demos.
- **Deferred design work** — Montoya integration, live Notion synchronization, additional module implementations, and a simultaneous GUI/CLI client mode remain future work.

5. AI collaboration record

OpenAI Codex, using a GPT-6.1 coding model and parallel agents, assisted with Java implementation, UI, parser hardening, model-response validation, verification, and report preparation. The user supplied the traffic-import concept, chose XSS as the first module in a wider architecture, and requested both GUI and CLI access. During implementation, Codex proposed the shared service boundary, a defensive model contract, and synthetic demonstration cases. Tests exercised the implementation and exposed integration issues. The development session did not provide verified token or billing totals.

Qwen2.5 0.5B Instruct is the runtime model used in verification. It ran through local Ollama. HuntAssist required no paid model API or API key for that run.

Representative interactions included the user's request for CLI parity, which led to a shared service contract, and the user's distinction between a first XSS module and the wider architecture, which led to a module interface and registry. In verification, the team traced a focused question to its action and citations, found that broad retrieval added irrelevant sections, and tightened topic filtering.

6. Development and verification

The Stage 2 instruction specifies three commit windows, October 20–26, October 27–November 2, and November 3–9, 2026 in Toronto. The repository history available at implementation time had zero meaningful implementation commits in those windows. The Stage 1 design commits and this implementation are dated November 11. The report records the actual dates. The stated Stage 2 deadline was November 10.

The first implementation commit added the shared Java services, both interfaces, database migrations, fixtures, and the Java tests. The later verification commit adds the report, demonstrations, and runnable KUMA suite. Implementation commit

The first build surfaced an ambiguous JDBC query callback. An explicit void callback resolved the overload. The final Java 21 verification passed 25 tests across eight classes. It exercised storage migrations, CLI response contracts, defensive capture parsing, note versioning, and model-response validation. The browser walkthrough exercises the packaged application with synthetic fixtures.

The final KUMA evaluation passed 11 cases, including two with the live local model. A pre-fix question returned the right CSP action alongside irrelevant citations. After restricting focused questions to matching note sections, the same case returned the CSP citation alone and preserved coverage. Stage 3 verification report

7. Demonstration index

The CLI recording uses the executable JAR and synthetic fixtures. Each segment shows a real command and its captured output. The browser demonstration uses the same services through the local web interface.

- **F01 Workspace and scope** — CLI 0:00 · Browser 0:00
- **F02 Traffic import and history** — CLI 0:06 · Browser 0:04
- **F03 Route inventory** — CLI 0:18 · Browser 0:08
- **F04 Defensive reference memory** — CLI 0:30 · Browser 0:20
- **F05 Pluggable XSS mitigation review** — CLI 0:42 · Browser 0:24
- **F06 Evidence-grounded remediation review** — CLI 0:48 · Browser 0:28
- **F07 Import-profile comparison** — CLI 0:24 · Browser 0:16
- **F08 Coverage and decisions** — CLI 1:00 · Browser 0:37
- **F09 Question-guided review** — CLI 0:54 · Browser 0:33
- **F10 Privacy, provenance, and audit** — CLI 1:06 · Browser 0:12